
Contextual information extraction of customer data from sales audio calls using a LoRA-fine-tuned language model

Thomas Hoang¹

¹Student ID: A1984077

Abstract

Sales agents on our CRM platform (uCall) currently fill in customer data fields by hand after every call, creating significant operational overhead. This project presents a complete, production-ready pipeline to automate this process. We chain Voice Activity Detection (Silero VAD) for removing background noise and a Vietnamese Automatic Speech Recognition model (wav2vec 2.0) for transcribing speech. For the final extraction stage, we demonstrate that a 0.8B parameter language model, fine-tuned using Low-Rank Adaptation (LoRA), can accurately extract structured CRM fields from these transcripts. Evaluated on a curated dataset of 550 real-world sales calls, the 0.8B model achieved high domain accuracy, outperforming a larger 2B model that collapsed under identical hardware constraints. Furthermore, we experimented across diverse hardware environments (Google Colab and Apple Silicon) to prove that businesses can deploy effective, localized AI solutions on consumer hardware, cutting the costs associated with large external cloud APIs.

Keywords Information Extraction, Large Language Models, LoRA Fine-Tuning, Automatic Speech Recognition, CRM Automation

Abbreviations LLM, LoRA, ASR, VAD, CRM

1. Introduction

1.1. Problem Statement

Sales agents on the uCall platform currently fill in customer data fields by hand after every call. This manual process takes too much time and causes errors. We are building an automated system to extract structured CRM fields directly from raw audio calls.

1.2. Motivation

Manual data entry costs businesses time and money. Large language models can extract this data but they are often too big and expensive to run. This project shows how to combine audio processing tools with a small local language model. We use a fine-tuning method called LoRA to accurately extract data using standard hardware. **This is important because it proves that companies do not need large external models to automate their data tasks.**

1.3. Contributions

- Designed a production-ready pipeline that solves a real-world business problem by processing raw audio into structured data.
- Experimented with fine-tuning Large Language Models across diverse hardware environments, including Google Colab (CUDA) and local Apple Silicon.

- Demonstrated that a fine-tuned 0.8B parameter model is highly effective and small enough to significantly cut operational LLM costs compared to expensive cloud APIs.

2. Related Work

Information extraction from spoken dialogue combines speech recognition and language modeling. Existing methods often rely on large language models for zero-shot extraction such as ChatGPT (9) or use separate large-scale speech models like Whisper (4). While these approaches are accurate, they are very expensive and slow for business use. Other research focuses on base architectures (8) or audio models like wav2vec 2.0 (1) and Zipformer (10). However, most studies look at these models in isolation. The gap in the current literature is a lack of end-to-end systems optimized for specific tasks on consumer hardware. Our approach combines a Vietnamese wav2vec 2.0 model with a small local language model. We use LoRA (3) to train this small model to perform extraction as well as larger models.

2.1. Novelty and Significance

This project is novel because it **builds a complete pipeline from raw stereo audio to structured data without relying on large external APIs**. We use Silero VAD (6) to clean the audio and maintain speaker channels. Then we **fine-tune a small local model to replace expensive cloud services**. The significance of this work is that it **provides a practical blueprint for companies**

to automate data entry. It shows that businesses can achieve high accuracy on domain specific tasks using local hardware.

3. Methodology

The goal of this work is automatically extracting customer information from call audio and fill the CRM fields. We proposing a three-stage pipeline to accomplish this, like illustrated in Figure 1.

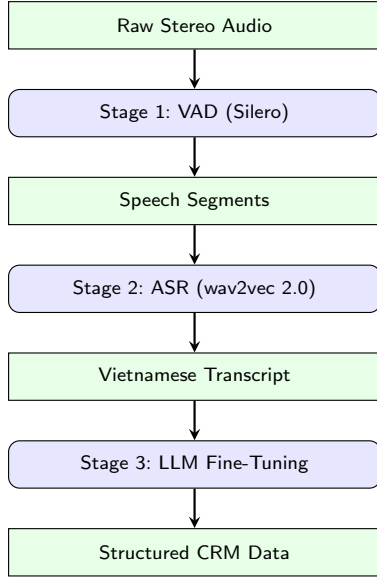


Figure 1 The three-stage contextual extraction pipeline.

3.1. Stage 1: Voice Activity Detection (VAD)

Raw files contain silence, hold tones, background noise, and non-speech events, which increase speech recognition errors. Preprocessing limits those segments. Recordings are stereo, with the agent on the left channel and the customer on the right. Since downstream analysis targets the customer, we isolate the right channel before VAD.

We reviewed comparative VAD evidence on mixed-domain audio at 16kHz (7). In that evaluation, **Silero v6** reports the best multi-domain ROC-AUC (**0.97**) and high chunk-level speech accuracy (**0.92**). Based on this quality evidence and deployment constraints, we adopted **Silero VAD** for this pipeline (6; 7). We retain the speech intervals and drop the rest, removing long silent regions and background sound prior to ASR. Figure 2 shows an example call before and after VAD processing.

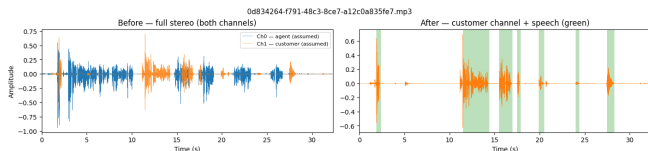


Figure 2 One call: stereo mix (left) and customer channel with VAD speech regions (right).

3.2. Stage 2: Automatic Speech Recognition (ASR)

After VAD segmentation, each speech-only chunk is forwarded to an Automatic Speech Recognition (ASR) model to convert audio segments into unstructured text transcripts. The segment transcripts are then merged by timestamp to reconstruct the full-call transcript.

Three ASR candidates were considered: wav2vec 2.0 with a Vietnamese checkpoint (2), Whisper (4), and Zipformer (5; 10). Ultimately, we chose the wav2vec 2.0 model (nguyenvulebinh/wav2vec2-base-vi) because it has already been running reliably in our production environment for 1.5 years.

In a real-world startup environment, practical constraints often matter just as much as theoretical benchmarks. Relying on a proven, already-integrated model avoids the technical debt and unexpected blockers that can come with migrating to an entirely new architecture. Because of this existing infrastructure, the pipeline implementation does not load the ASR model locally. Instead, it delegates the transcription task by making calls to our internal, containerized API that serves the model.

3.3. Stage 3: LLM Fine-Tuning

In the final stage, reconstructed transcripts are formatted into a supervised fine-tuning dataset for a downstream large language model. The model is trained to extract a comprehensive set of customer information in a single pass, outputting a structured JSON object. The target CRM schema contains six specific fields, as detailed in Table 1.

Table 1. CRM JSON Field Schema Extracted by the LLM

Field Name	Data Type	Description
customer_sector	String Null	Industry or sector
customer_needs	String	Brief summary of what the customer needs
customer_interested	Number	1 to 10 scale of interest
customer_busy	Boolean	True if they said they are busy, false otherwise
customer_scheduled_a\$	String Null	Time or date they want to be called back
customer_rating	Number Null	Optional 1 to 10 rating, if available

Of the initial 600 selected calls, 50 records were dropped because they failed to generate a transcript during the ASR stage. The remaining dataset is compiled into 550 rows of JSON-formatted training examples for supervised fine-tuning. Each row contains the instruction (the extraction prompt and schema), the input (the full call transcript), and the response (the exact JSON output with all six fields populated). A representative training example is shown below (the transcript and response are formatted for readability):

```

{
  "call_id": "03bc8c4e...",
  "instruction": "You are a structured-field

```

extractor for telesales CRM QA. Read the following transcript and extract the customer information.

Output valid JSON matching this schema exactly:

```
{
  \"customer_sector\": \"string\",
  \"customer_needs\": \"string\",
  \"customer_interested\": \"number (1-10)\",
  \"customer_busy\": \"boolean\",
  \"customer_scheduled_at\": \"string or null\",
  \"customer_rating\": \"number (1-10)\"
}
```

Return only JSON. Do not include markdown.",
: "[Full Vietnamese call transcript...]",
 response: "[Extracted values matching schema]"

Model Architecture and Loss

Candidate model families for extraction initially included:

- Qwen/Qwen3.5-2B: Evaluated successfully.
- Qwen/Qwen3.5-0.8B: Evaluated successfully.
- google/gemma-4-E4B: Excluded from final evaluation (failed during training).

All are decoder-only Transformer models (8) built on self-attention:

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^\top / \sqrt{d_k}) V,$$

which allows tokens to attend to the full transcript to resolve implicit context. Smaller models are preferred to reduce inference and training costs.

Because updating every parameter of the model is computationally expensive, we employ Low-Rank Adaptation (LoRA) (3). The pre-trained weights are frozen and only a low-rank update is trained, cutting trainable parameters to a small fraction. We train using the standard autoregressive cross-entropy loss over the response tokens:

$$\mathcal{L} = -\frac{1}{|\mathcal{R}|} \sum_{t \in \mathcal{R}} \log p_\theta(y_t | y_{<t}) \quad (1)$$

where $\mathcal{R}$ is the set of response-token positions. Minimising this loss teaches the model to assign high probability to the exact labels defined by the CRM field configuration.

4. Experimental Setup

4.1. Data

The dataset is extracted from the internal staging database of our organization, consisting of real manual sales calls made through the CRM system. We selected calls with a duration greater than 5 seconds to exclude empty or dropped connections. The full export contains 13,232 stereo audio records. For the purposes of evaluating the extraction pipeline in this project, an initial subset of 600 calls was selected. During processing, 50 records were dropped due to being broken (failing to yield a transcript), resulting in a final curated dataset of 550 calls. This dataset is partitioned into an 80/20 train-test split (440 train, 110 test) for supervised fine-tuning.

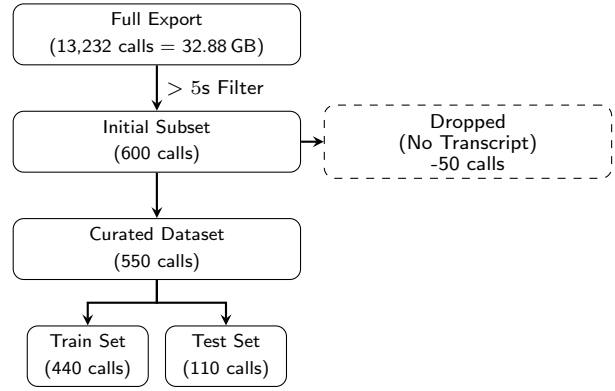


Figure 3 Dataset curation and train-test splitting process.

4.2. Hardware and Software

The pipeline is executed across two distinct environments to balance local prototyping and accelerated training:

- **Local Preprocessing (Apple MacBook M4):** Used for the initial data preparation stages, including stereo audio isolation, Voice Activity Detection using Silero VAD (v6.2.1), and assembling the final JSON training dataset.
- **Cloud Training (Google Colab T4):** The computationally intensive supervised fine-tuning (Stage 3) is conducted in the cloud using an NVIDIA T4 GPU (16 GB VRAM).
- **Software Stack:** The environment is managed via Conda on Python 3.11.15. Key libraries include PyTorch (v2.11.0) for tensor operations, the Hugging Face transformers library (v5.5.0) for model architectures, and Unsloth for LoRA training.

4.3. Hyperparameters

To ensure reproducibility, the hyperparameter configuration used for the supervised fine-tuning of the Qwen models is provided in Table 2. We utilized 4-bit quantization alongside Low-Rank Adaptation (LoRA) (3) to optimize memory efficiency and fit the training workload within the 16 GB limit of the T4 GPU.

Table 2. Model Fine-Tuning Hyperparameters

Hyperparameter	Value
LoRA Rank (r)	16
LoRA Alpha (α)	16
LoRA Dropout	0
Learning Rate	2×10^{-4}
Per-Device Batch Size	2
Gradient Accumulation Steps	4
Effective Batch Size	8
Epochs	1
Max Sequence Length	2048 tokens
Quantization	4-bit (NF4)

These specific hyperparameters were chosen to balance model performance against strict hardware limitations and data constraints:

- **Epochs (1):** Given the relatively small size of the curated dataset (440 training records), limiting training to a single epoch helps prevent the model from catastrophically overfitting and memorizing the training transcripts.
- **Batch Size (2) & Gradient Accumulation (4):** A small per-device batch size was strictly necessary to avoid Out-Of-Memory (OOM) errors on the 16 GB T4 GPU. Gradient accumulation is used to simulate a larger effective batch size of 8, which stabilizes the training gradients.
- **LoRA Rank ($r=16$):** A rank of 16 provides an optimal trade-off between expressivity and memory overhead. It introduces enough trainable parameters to help the model learn the complex JSON extraction schema without inflating VRAM usage.

5. Results

5.1. Training Convergence

Figure 4 illustrates the training loss for both the Qwen3.5-0.8B and Qwen3.5-2B models across the 55 training steps. Both models demonstrate clear convergence, rapidly adapting to the JSON schema format and stabilizing towards the end of the single epoch.



Figure 4 Training loss curves for Qwen3.5-0.8B and Qwen3.5-2B over one epoch (55 steps) of LoRA fine-tuning.

5.2. Extraction Performance

To ensure a fair evaluation, the metrics used to score the model’s outputs were tailored to the data type of each specific CRM field, as detailed in Table 3.

Table 3. Evaluation Metrics by Extracted Field

JSON Field	Evaluation Method
Overall Format	Strict JSON Parsing (Pass/Fail)
customer_busy	Exact Match (Boolean)
customer_sector	Soft-Matching (Substring)
customer_scheduled_at	Soft-Matching (Substring)
customer_interested	Mean Absolute Error (MAE)

Table 4 details the extraction performance across the 110-call test set. Key observations include:

- **0.8B Model Success:** The fine-tuned Qwen3.5-0.8B model demonstrates massive improvement in domain-specific tasks, such as customer_sector (jumping from 8.18% to 41.35% accuracy) and customer_scheduled_at (improving to 75.96%).
- **2B Model Failure:** Interestingly, the Qwen3.5-2B model showed no change between its base and fine-tuned states, performing poorly overall compared to the smaller 0.8B variant.
- **Environment Constraints:** The exact cause of the 2B model anomaly is unconfirmed, but it is highly suspected to be an environment compatibility issue. Because cloud computing credits were exhausted after training on the Colab T4 GPU, the final evaluation was executed locally on Apple Silicon (MacBook M4). This architectural transition likely caused a silent failure when attempting to merge or load the 4-bit LoRA adapter weights onto the base model using macOS.

6. Discussion

6.1. Obstacles and Strategies

The reality of executing this pipeline exposed significant gaps between theoretical ML workflows and practical constraints. Our main engineering challenges revolved around data processing, hyperparameter tuning, and flawed evaluation setups:

- **Data Leakage and Train/Test Partitioning:** During our initial modeling phase, we make a critical oversight: we forgetting explicitly partition the dataset into training and testing sets before fine-tuning. When we first evaluating the model, it achieve suspiciously perfect accuracy on the demo queries. Realizing we had severely overfit the model via data leakage, we had to scrap the LoRA adapter entirely, implement a strict 80/20 data split, and retrain from scratch to obtain honest metrics.
- **Hyperparameter Tuning Instability:** We spent considerable time experimenting with LoRA hyperparameters, which initially led to catastrophic results. We mistakenly set the learning rate too high ($1e-3$) while pushing the adapter rank to $r = 64$ in an attempt to force the model to learn faster. This caused severe training instability and loss spikes, resulting in a model that output repetitive gibberish. We overcame this by lowering the learning rate to $2e-4$ and aggressively reducing the rank to $r = 16$, which immediately stabilized convergence.
- **Evaluation Framework Limitations:** Choosing how to evaluate the model proved to be one of our most frustrating obstacles. Standard classification metrics completely failed because generative text doesn’t always match predefined labels exactly. Our initial Python evaluation scripts incorrectly penalized the model for minor formatting differences (e.g., "IT_Support" vs. "IT Support"). We had to completely overhaul our evaluation strategy mid-project, pivoting to semantic soft-matching and Mean Absolute Error (MAE) to fairly assess the model’s actual intent.
- **Data Engineering Overhead:** We significantly underestimated the overhead of processing the raw 32 GB audio dataset. Our

Table 4. Quantitative Evaluation on the 110-Call Test Set

Model Configuration	Valid JSON (%)	Busy Acc (%)	Sector Match (%)	Schedule Match (%)	Interested (MAE)
Qwen3.5-0.8B (Base)	100.00	64.55	8.18	70.00	4.07
Qwen3.5-0.8B (Fine-Tuned)	94.55	56.73	41.35	75.96	3.68
Qwen3.5-2B (Base)	76.36	30.95	0.00	34.52	3.62
Qwen3.5-2B (Fine-Tuned)	76.36	30.95	0.00	34.52	3.62

initial sequential download script was projected to take over 24 hours. We had to pause modeling entirely to rewrite the data pipeline using Python’s `concurrent.futures`. While this reduced ingestion time to a few hours, it immediately introduced race conditions and API rate-limiting blocks from Gemini, forcing us to implement exponential backoff.

- **Cross-Platform Architectural Compatibility:** A major oversight was failing to anticipate the architectural friction between our development environment (macOS) and our training environment (NVIDIA T4). We trained the adapters using PyTorch/CUDA in the cloud. However, attempting to load the PyTorch-saved adapters into MLX locally resulted in silent crashes (e.g., `SimpleNamespace` attribute errors). This forced us into a complex process of manually translating adapter weights just to run inference.
- **Hardware Constraints and Out-of-Memory Errors:** We hit a hard ceiling on the Colab 16GB T4 GPU when attempting to train the Qwen3.5-2B model. To salvage the run, we halved the `per_device_train_batch_size` to 1 and doubled the `gradient_accumulation_steps` to 8, allowing the optimizer to step without crashing. The larger `gemma-4-E4B` model had to be abandoned entirely due to VRAM exhaustion.

6.2. What We Learned

Beyond the code, this project fundamentally shifted our understanding of applied machine learning:

- **Complexity of Generative Evaluation:** Our struggles with the evaluation pipeline proved that defining quantitative metrics for generative LLM outputs is an iterative, ambiguous process. We learned that standard metrics are too strict for language tasks, and that building a robust, fault-tolerant evaluation script is just as critical as training the model itself.
- **Data Quality Versus Parameter Scale:** The failure of the 2B model compared to the 0.8B model was highly instructive. We initially assumed a larger model would naturally capture more nuance. Instead, the 2B model suffered from optimization collapse because we had to squeeze it so aggressively to fit into VRAM. It proved that a very small, well-optimized model (0.8B) trained on clean, highly curated data will always beat a larger model that is starved of memory and compute.

6.3. Future Directions

If provided with unlimited compute and time, we would evolve this project in three specific directions:

- **Dataset Scaling and Manual Annotation:** We were constrained to a 550-call subset purely by API costs and time. With unlimited resources, we would process all 13,232 calls. More importantly, we would replace the Gemini-generated silver labels with a manual human review system to create a genuine gold-standard dataset, eliminating the current ceiling caused by the teacher model’s inherited biases.
- **End-to-End Latency Optimization:** Currently, the pipeline is entirely asynchronous (Audio → VAD → ASR → LLM). A major research path would be optimizing the `wav2vec 2.0` ASR and Qwen extraction models into a single, streaming multimodal architecture that could extract CRM fields in real-time while the telemarketer is still on the call.
- **Automated Evaluation Frameworks:** To solve our evaluation bottlenecks, we would implement an automated grading framework. Instead of relying on Python regex scripts, a larger frontier model would read the transcript and the student model’s JSON output, semantically scoring whether the extracted fields accurately reflect the dialogue context.

7. Conclusion

This project successfully demonstrate the viability of automating structured CRM data extraction from noisy Vietnamese telemarketing calls using a fully localized, resource-constrained pipeline. By sequentially chaining Voice Activity Detection (Silero VAD), Automatic Speech Recognition (`wav2vec 2.0`), and a Low-Rank Adaptation (LoRA) fine-tuned language model (Qwen3.5-0.8B), we proved that complex, domain-specific semantic extraction can be achieved without relying on large proprietary cloud APIs.

Our findings indicate that in the realm of applied generative AI, high-quality data curation and architectural alignment is heavily outweighing raw parameter count. The success of the instruction-tuned 0.8B model, contrasted against the optimization collapse of the larger 2B model on identical 16GB hardware, underscores the necessity of matching model scope to physical computing constraints. Ultimately, this pipeline provides a foundational blueprint for enterprises seeking to reduce the operational overhead of manual data entry while maintaining complete internal data sovereignty and cost efficiency.

References

1. Alexei Baevski, Yuhao Zhou, Abdelrahman Mohamed, and Michael Auli. `wav2vec 2.0`: A framework for self-supervised learning of speech representations. *Advances in Neural Information Processing Systems*, 33:12449–12460, 2020.

2. Nguyen Vu Le Binh. wav2vec2-base-vi. <https://huggingface.co/nguyenvulebinh/wav2vec2-base-vi>, 2022. Vietnamese self-supervised pre-trained wav2vec2.
3. Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models, 2021.
4. OpenAI. Whisper: Robust speech recognition via large-scale weak supervision. <https://github.com/openai/whisper>, 2025. GitHub repository.
5. Qualcomm. Zipformer. <https://huggingface.co/qualcomm/Zipformer>, 2025. Model card and deployment assets.
6. Silero Team. Silero vad: pre-trained enterprise-grade voice activity detector (vad), number detector and language classifier. <https://github.com/snakers4/silero-vad>, 2024. GitHub repository.
7. Silero Team. Silero vad quality metrics. <https://github.com/snakers4/silero-vad/wiki/Quality-Metrics>, 2026. Includes multi-domain ROC-AUC, accuracy tables, and model-author comparison notes.
8. Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
9. Xiang Wei, Xingyu Cui, Ning Cheng, Xiaobin Wang, Xin Zhang, Shen Huang, Pengjun Xie, Jinan Xu, Yufeng Chen, Meishan Zhang, Yong Jiang, and Wenjuan Han. Zero-shot information extraction via chatting with ChatGPT. *arXiv preprint arXiv:2302.10205*, 2023.
10. Zengwei Yao, Liyong Guo, Xiaoyu Yang, Wei Kang, Fangjun Kuang, Yifan Yang, Zengrui Jin, Long Lin, and Daniel Povey. Zipformer: A faster and better encoder for automatic speech recognition. *arXiv preprint arXiv:2310.11230*, 2024.